Decoding Crime Narratives using NLP and Big Data Analytics

A complete beginner's project report — Version 3.1

The 3.1 entity revision adds generated ownership, active/passive violence and FIR-style administrative contexts to CRF/BERT training, fixes shared punctuation boundaries and preserves exact displayed characters. See ENTITY_CONTEXT_V3_1.md. The NER evaluation now mixes a previously viewed V3 subset with generated context examples; its scope differs from the unchanged document-model benchmark.

Version 3 adds informal English and spelling variation within the original algorithms. Read ROBUSTNESS_V3.md for the precise changes, development challenges and limits. The measured appendix contains the current results; the explicitly labeled V2 discussion is historical.

The schema-4 generalization upgrade retains that core and adds separately trained organization/witness recognition, full-document retrieval and BERT windows, qualified action/reference notes, entity corrections, and a human annotation/quality-review workflow. See UPGRADE_STATUS.md. Its synthetic diagnostics are separate from the legacy benchmarks; real-data validation and human quality judgments remain pending.

How to read this report

Start at Chapter 1 even if you have never used machine learning. Chapters 1–4 explain the problem and the data. Chapters 5–9 explain every NLP component with examples. Chapters 10–13 explain evaluation, the dashboard and how to run the code. Chapters 14–17 cover the syllabus, limitations, improvements and glossary. The measured-results appendix is generated from the actual saved experiment outputs, rather than manually typed scores.

This is an English-language university research prototype. Its 60,000 clean reports describe fictional events. It demonstrates engineering and NLP methods; it does not establish that its predictions are accurate on real police files.

1. The problem in ordinary language

A narrative is a description written as sentences. Reading thousands of descriptions manually makes it difficult to count incident categories, find mentions of a weapon, or discover repeated themes.

This project turns a narrative into searchable, structured information. For example, consider this invented illustration:

> On 2025-03-18 at 21:15, Alex Lee reported a stolen phone near Cedar Market. A witness identified Morgan Reed as a suspect. No knife was reported. Officers reviewed camera footage.

The system can suggest a crime category, highlight participants and locations, identify dates and evidence, display sentence grammar, retrieve relevant cases, and create a short extractive summary.

The example illustrates concepts; it is not a claim that every model returns these exact answers. A suspect mention represents a role in an allegation, not a finding of guilt. The phrase “No knife” must not become evidence that a knife was used.

What goes in and what comes out?

Input	Operation	Output
Text descriptions	Spark validation and cleaning	Unique valid reports
Clean training text and category labels	Train document classifiers	Suggested category
Training text with marked character spans	Train sequence taggers	Entity mentions and roles
Unlabeled training text	Topic modeling	Recurring word themes
An individual report	spaCy, VADER and TextRank	Grammar, polarity and summary
A search query	Keyword or embedding retrieval	Ranked reports and source sentences

2. A map of the entire system

```
Fictional report generator
    -> raw JSONL: one JSON object per line
    -> Apache Spark: validate, normalize, deduplicate
    -> partitioned Parquet: clean analytical storage
    -> fixed train / validation / development / final-test groups
    -> fit models on training examples
    -> select checkpoints and calibrate on validation
    -> freeze model hashes
    -> evaluate the final synthetic test
    -> enrich all reports using saved models
    -> SQLite case store + keyword index + semantic index
    -> Streamlit dashboard
```

A **pipeline** is this ordered sequence of steps. Each step produces files that the next step reads. Training teaches a model; inference uses a trained model to analyze new text. Opening the dashboard performs inference, not training.

Most expensive processing happens before the browser opens. This is a batch system, not a live police-feed service.

3. Dataset: where the reports come from

The project deliberately generates its own dataset, which is allowed by the project requirements. It does not silently substitute synthetic text for an alleged real-world download.

There are eight balanced categories: Assault, Arson, Burglary, Cybercrime, Fraud, Robbery, Theft and Vandalism. Events and roles are fictional. Place vocabulary includes real international cities; no event is a claim about those cities or any actual person. The generator produces 60,000 valid unique examples, then injects known faults so cleaning can be checked:

Injected condition	Rows	Why include it?
Duplicate reports	1,500	Check deduplication
Null narratives	600	Check missing-value rejection
Blank narratives	300	Check whitespace validation
Invalid dates	300	Check date parsing
Total raw rows	62,700	Input to Spark

Version 3 includes **528 scenario groups**: 384 formal composite groups and 144 informal event groups. There are 12,000 formal reports and 48,000 informal reports. Informal inputs include short fragments, Indian English phrasings, actual misspellings, omitted function words, irregular spacing and varied casing. These are generated groups, not independently authored stories. Many words and grammar patterns remain shared.

Formal participant phrases and first-name pools differ by split. Informal event patterns, full-name pools and place pools differ by split; randomly invented names add unfamiliar surfaces. All noise variants of an event remain in its assigned split. Normalized duplicates are rejected across all splits. Some formal stories also have unidentified suspects, multiple participants and negated weapons. Original source spans remain exact through every training-text edit.

A record and its fields

A JSON object is a set of named values. JSONL stores one such object per line. The generated record includes:

Field	Meaning
report_id	Stable identifier used to find a case
narrative	Original text, preserved for display and offsets
crime_type	Generator's document-category label
district, reported_at	Fictional location grouping and timestamp
template_group, split	Scenario family and assigned evaluation partition
entities_json	Gold entity labels and character positions

“Gold” means the reference annotation used for scoring. Here it is created by the generator, not a human annotator. For “Alex Lee,” a span could be start 0, end 8. Python uses an exclusive end: `text[0:8]` returns the eight characters.

Preserving original text matters: removing characters before NER would shift these positions and highlight the wrong words.

External data import

`scripts/import_jsonl.py` validates a separately supplied JSONL file, checks IDs, dates, text length and optional entity offsets, and records source URL, license and SHA-256 hash. Invalid rows are reported by line number and reason. The importer writes to an ignored staging directory.

It does **not** automatically mix external data into the benchmark or train on it. A real-data experiment needs permission, a label mapping, reviewed annotations and a separate split design first. The source URL and license arguments document provenance; entering them does not itself prove that use is permitted.

4. Big Data processing with Apache Spark

The main Spark stage is now implemented in Scala in `scala/CrimePipeline.scala`. Python orchestrates compilation/submission and retains the NLP models and UI. The original PySpark implementation remains available with `--spark-engine pyspark`. See Scala execution and verification; historical cluster and model receipts describe the runs that originally produced them.

Why Spark?

A normal Python script often processes a dataset in one process. Spark divides a DataFrame into **partitions** and schedules work on executors. A DataFrame resembles a table with named columns and known types.

A **driver** plans the work. A **master** coordinates worker resources in standalone mode. A **worker** hosts executor processes. Executors run tasks over partitions. An **action**, such as counting or writing data, triggers execution; transformations such as filtering describe the work to do.

This project is a modest corpus demonstrating distributed techniques. Sixty thousand reports do not require a production-scale cluster.

Cleaning steps

1. Read JSONL using an explicit schema rather than guessing all column types.
2. Reject missing or blank text and dates that cannot be parsed.
3. Keep original text and create a normalized lowercase text column.
4. Calculate a SHA-256 text hash and remove repeated normalized narratives.
5. Validate split assignments and retain one record for each duplicate group.
6. Write partitioned Parquet and aggregate counts.
7. Train and evaluate a distributed MLlib classification baseline.

Parquet is a column-oriented file format. Reading only the columns needed for a chart can reduce I/O compared with loading every long narrative. Partition folders organize reports by their split.

A **shuffle** redistributes records across executors. Deduplication and grouped aggregation may require shuffles; they can be more expensive than filtering a partition independently.

What was actually executed?

Version 3 was run against a standalone master and **two worker processes on this one Windows computer**. The run processed all 60,000 clean reports. `artifacts/cluster_execution.json` records worker registration and completion; `spark_metrics.json` records the actual master and counts. `spark_execution_plan.txt` exposes the physical plan.

This is separate-worker execution on one host, not proof of multi-machine scalability, fault tolerance under machine failure, or an HDFS deployment. The default simpler reproduction uses `local[2]`, which runs local Spark tasks without a separate master.

MLlib baseline

The MLlib pipeline is:

```
RegexTokenizer -> StopWordsRemover -> HashingTF -> IDF -> LogisticRegression
```

It converts text into numerical features, learns a category classifier, and evaluates a held-out partition. It uses more training and test rows than the four-model Python comparison. Therefore its score is informative but not an equal-sample model comparison.

Hadoop is an ecosystem; HDFS is distributed storage; MapReduce is a map/shuffle/reduce processing model. Spark can work with Hadoop storage, but Spark and Hadoop are not the same product. Here storage is local Parquet. HDFS, YARN, MongoDB and Kafka are not running services.

5. Preprocessing, grammar and morphology

Tokenization divides text into words or subword pieces. **Lemmatization** maps inflections to dictionary forms: “reported” becomes “report.” **POS tagging** labels grammatical roles such as noun, verb and adjective. **Morphology** describes features such as tense, number or verb form.

A **dependency parse** links a word to the word it depends on. In “Alex reported a theft,” “reported” may be the root verb, “Alex” its subject, and “theft” its object. spaCy predicts these relationships. The dashboard draws arrows and also lists tokens, lemmas, POS tags, morphology and heads.

Different tasks need different preprocessing. Document vectorizers tokenize and normalize internally. Topic modeling uses selected noun, verb and adjective lemmas and suppresses names and common reporting words. NER retains word shapes, capitalization and character offsets because they help identify names.

The spaCy model is pretrained. The project does not train a dependency parser or report a syntax accuracy score without a gold syntax test set.

6. How words become numbers

Machine-learning models consume numerical representations. This project compares three representation families on the same document split.

6.1 Bag of Words

Suppose the vocabulary is [phone, stolen, fire]. “Stolen phone” becomes [1, 1, 0]; “fire fire” becomes [0, 0, 2]. This representation counts words and largely ignores order. Bigrams add adjacent pairs such as “stolen phone,” giving limited local order information.

The implementation uses CountVectorizer and logistic regression. Logistic regression learns a weight for each feature and class, then converts scores into class probabilities. Validation tuning selects binary word/phrase presence, L2 normalization and C=10 for the current Bag of Words model; the original baseline used raw counts.

6.2 TF-IDF

Term Frequency–Inverse Document Frequency gives more weight to words that distinguish a document from the rest of the training corpus. A word appearing in nearly every report usually provides less information.

A common smoothed inverse frequency is:

```
IDF(word) = log((1 + number_of_documents) / (1 + documents_containing_word)) + 1
```

In a corpus of 100 documents, a word appearing in 99 has IDF about 1.01; a word appearing in 4 has IDF about 4.01. Actual feature values also depend on the vectorizer's term-frequency and normalization settings. These numbers are a teaching illustration, not saved experiment features.

The vocabulary and IDF are fitted on training text only. Fitting them on the test set would leak information about the evaluation distribution.

Version 3 combines word unigrams/bigrams with character 3–5-grams in the same TF-IDF representation and logistic regression model. A typo can lose its whole-word match while retaining useful character fragments. Character features received weight 0.75 in the baseline; validation tuning selects weight 1.5 and logistic-regression C=0.25 for the current model. This does not autocorrect the source and does not introduce a new classification algorithm. Keyword search continues to use the word branch so a misspelled string cannot become an unexplained exact-keyword match.

6.3 Neural Word2Vec embeddings

Word2Vec is a small neural model trained to learn contextual relationships between words. This project uses Gensim skip-gram with 100-dimensional vectors, trained on its training partition. The deployed baseline averages known word vectors. A validation-selected candidate concatenates an IDF-weighted mean with max pooling, producing 200-dimensional document features. Training-only IDF emphasizes less common words; L2 normalization and logistic regression follow. The candidate improved validation but fell to 75.75% accuracy on the viewed diagnostic test, compared with 82.17% for the baseline. Its experiment is preserved and deployment retains the baseline. This release rollback uses diagnostic test results; it is not an untouched final evaluation, and no further tuning followed that result.

Pooling is efficient but loses word order. “Alex accused Morgan” and “Morgan accused Alex” can have similar pooled vectors even though their roles differ. Unknown words contribute no vector; an entirely unknown document gets a zero vector.

6.4 Contextual BERT representations

BERT uses transformer attention so a word's representation depends on surrounding words. The project fine-tunes a compact pretrained BERT for document classification. Training updates model parameters; a randomly initialized category head alone is not a useful pretrained crime classifier.

The checkpoint is selected using validation macro-F1. A larger neural model is not automatically better than a simple word-count model, especially on repetitive synthetic data with a small training sample.

The archived tuned BERT document classifier uses a 192-subword truncating limit, compared with 160 in the baseline. The live schema-4 mode now covers the full document with overlapping windows and averages their predictions. A separately trained windowed candidate did not beat validation and was retained only as an experiment; the active mode uses the original weights. BERT entity extraction already uses overlapping word windows. The original benchmark files remain unchanged. See UPGRADE_STATUS.md and ACCURACY_IMPROVEMENTS.md for the separate recipes and evaluation scopes.

7. Sequence labeling: finding entity roles

Document classification assigns one category to a complete narrative. Sequence labeling assigns a label to each token.

Eight entity types are supported: SUSPECT, VICTIM, LOCATION, WEAPON, PROPERTY, DATE, TIME and EVIDENCE.

A disclosed media-evidence context check corrects clear footage/recording mentions in collection/review contexts after model prediction. For example, “Camera footage was collected” is EVIDENCE. These corrections retain exact source offsets, record their extraction method and have no model probability. Bare stolen cameras remain property. The existing learned model weights and raw benchmark metrics are retained; see EVIDENCE_CONTEXT_FIX.md for regression and non-regression verification.

BIO notation

Token	Label
Alex	B-VICTIM
Lee	I-VICTIM
reported	O
a	O
phone	B-PROPERTY

B means beginning, I means inside the same entity, and O means outside an entity. Both span boundaries and entity type must be correct for strict entity scoring.

CRF

The core context layer separates named witness clauses from locations and checks clear syntactic participant/ownership roles. Its repairs have no model probability and do not alter the retained core weights. The schema-4 specialist now also supplies learned ORGANIZATION/WITNESS mentions with separate scores; witnesses remain stored separately for compatibility. The older context layer has its own measured scores, including a small CRF validation regression, in `artifacts/narrative_context_validation.json`. See NARRATIVE_VALIDATION.md for the two reported cases, development audit and cross-checks; the earlier media-only receipt does not measure this broader layer.

A Conditional Random Field learns relationships between nearby labels and features such as capitalization, neighboring words, position and sentence context. It can learn that a name after a reporting phrase often describes a victim, while a name after an identification phrase may be a suspect. These are learned patterns, not guaranteed rules.

Version 3 uses a four-token lexical window on each side plus word shape and distances to action, role and preposition cues within eight tokens. Training includes lowercase and invented names, active/passive constructions and varied name lengths. Cue observations do not directly assign roles: the same linear-chain CRF learns their weights. No inference-time name or country list is used.

BERT token classifier

A second model fine-tunes BERT to predict BIO labels. BERT may split one word into multiple subwords; only the first subword receives a training label and other pieces use the ignored loss value -100. Predictions are mapped back to original words.

Overlapping windows reduce boundary losses in long cases. Probabilities for repeated words are averaged, and invalid orphan I-tags are repaired to begin a span. Extremely unusual text can still challenge tokenization.

Which model is shown?

Both models are trained and scored on validation. The higher validation strict span F1 wins; ties prefer the faster CRF. The current selection is saved in the model folder and shown in the dashboard. Users can explicitly compare both in Narrative Lab. Model choice is not changed after inspecting final-test scores.

A short preceding-word heuristic marks mentions such as “No knife” as negated. It is not full logical reasoning and may mishandle complicated negation or quoted allegations. Entity confidence is a model score, not verified factual confidence.

8. Topics, sentiment and summaries

Topic modeling: LDA and NMF

A topic is a group of words that often occur together. Latent Dirichlet Allocation represents a document as a mixture of topics. A report could contain both a property theme and a digital-access theme. Topic numbers are arbitrary identifiers, not crime labels.

Training uses lemmatized content words and reduces proper names and routine reporting vocabulary. LDA's top words, document-topic probabilities, held-out perplexity, UMass coherence and top-term diversity are saved. NMF provides a second unsupervised factorization and top-term comparison.

Lower perplexity indicates better fit under a fixed vocabulary and protocol; it does not guarantee more meaningful topics. Coherence measures word co-occurrence; diversity measures how many top words are distinct. Human review is still needed to decide whether themes are useful. NMF is not presented as a fully evaluated replacement selected by human topic judgments.

VADER sentiment and threat-language cues

NLTK VADER produces a compound polarity score between -1 and +1. Negative language is not equivalent to high danger. A neutral report can describe a serious event, and frightened wording can appear in a low-severity event.

The dashboard labels the unchanged VADER score as Negative (at or below -0.05), Neutral (between -0.05 and +0.05), or Positive (at or above +0.05). Analysis details show the three polarity components, scores for original source sentences, and base sentiment-lexicon entries. The component shares are not confidence probabilities. The word-level entries precede VADER's negation, intensifier and contrast rules and must not be read as final contributions. A failed attack can still have negative wording; negating a harmful word can also yield a positive lexical score without establishing happiness, safety or a positive event.

For killing-related wording outside the eight trained classes, a separate source-based syntax description distinguishes intent, threat, attempt, prevention, negation and completed wording. For example, “Mayank had ill intentions towards Barkha, and he went to her cabin to kill her, but the attempt failed” is displayed as **Attempted killing**, with **Attempt failed** as its outcome, and **Negative (-0.8481)** as its text polarity. The raw TF-IDF and BERT suggestions remain available in Analysis details. This is an explicitly disclosed context heuristic, not a newly trained homicide model or a legal classification. See the incident and sentiment update for its contract and limitations.

A separate transparent heuristic counts selected unnegated threat-language cues. It is shown as a demonstration cue level, not a probability of future violence or an operational risk assessment. There is no labeled danger dataset in this project.

TextRank summarization

TextRank treats sentences as nodes in a graph, connects similar sentences, and ranks them using PageRank. The implementation also includes a small lead-sentence preference. The application selects one source sentence for each case and normalizes its whitespace to one line. The original narrative is unchanged, and structured notes retain the original sentence citation. These summaries are stored in the enriched dataset, shown in case search, included in CSV exports, and computed on demand for new narratives. A long source sentence can still produce a long summary.

This is **extractive** summarization. It does not generate a new account. It can still omit an important denial or overemphasize repeated material. Structured notes list extracted mentions and source sentence references; users should check them against the narrative.

No ROUGE score is claimed because human reference summaries have not been collected.

9. Search, evidence retrieval and explanation

Keyword search uses TF-IDF cosine similarity. It works well when query and report share terms. Semantic search uses the pretrained all-MiniLM-L6-v2 encoder, attention-mask mean pooling and normalized 384-dimensional embeddings.

The legacy case embedding used only the first 128 subword tokens. Live retrieval now indexes every source window with 128-subword limits and 32-subword overlap: 61,300 windows cover the 60,000 reports. Ranking takes the best window match per report. Source offsets, report order and text fingerprints are checked. The float16 index uses memory maps and block scoring. Evidence retrieval also scans later windows of long sentences; relevance quality still needs human judgments.

Hybrid search combines keyword and semantic **ranks**, using reciprocal-rank fusion. It does not pretend that the two raw similarity scales are identical. Entity filters use predicted mentions stored in SQLite; missed entities can therefore cause missed filtered results.

For a question within a case, the app retrieves up to two matching source windows and displays the report ID and sentence numbers. A window qualifies when it contains every non-stopword query token as a whole word (case-insensitive), or its semantic similarity is at least 0.30. Keyword matches take priority, so a short query such as “gun” is retained even when its embedding similarity to a longer sentence is low. If neither route matches, the app returns “Insufficient evidence”. The UI identifies keyword and semantic matches and preserves the source wording, including negation. This is evidence retrieval, not a general conversational assistant or verified question-answering model. A retrieved sentence may be relevant without answering the question.

TF-IDF prediction explanations show positively contributing words, phrases and character fragments, with their feature type. They describe correlations in a linear classifier, not causal reasons why a crime occurred.

10. Evaluation: how to judge the results honestly

Four data roles

Split	Use	V3 sample for document models
Training	Fit parameters and feature vocabulary	12,000
Validation	Select checkpoints and fit temperature	800
Development	Separate diagnostic reporting	800
Final test	Score frozen models	1,200

The complete corpus has larger split partitions. These samples keep the CPU experiment practical. NER uses its own documented subsets: 8,000 training reports, 400 validation reports and 600 final-test reports.

Incident variants, participant phrasings and first-name pools are assigned before fitting. Composite groups do not overlap between splits. Three-fold grouped cross-validation of TF-IDF uses incident variants within training, keeping related layouts together.

A SHA-256 receipt records model files before final predictions. The final-test receipt records completion. Once results have been viewed, the test is no longer unseen to the researchers; future tuning should use a new independent evaluation protocol.

The current rebuild checks cross-split report IDs, scenario groups and normalized text before fitting, and records split fingerprints and source hashes. It saves row-level document and entity predictions. Run `python scripts/verify_evaluation.py` to check frozen model/prediction hashes, source labels and entity spans, and recompute Accuracy, macro/weighted F1 and strict Span-F1 without retraining. This remains a repeat synthetic evaluation on previously viewed test data; the additional audit checks do not make it an untouched benchmark. See IMPLEMENTATION_STATUS.md for the requirement mapping and local deployment commands.

The corpus still shares generator vocabulary and grammar across splits. This protocol reduces some leakage but cannot establish real-world generalization.

Metrics with small examples

If 80 out of 100 report categories are correct, accuracy is 80%.

If a model marks 10 names as victims and 8 are correct, precision is 8/10. If 12 actual victim mentions existed, recall is 8/12. F1 balances precision and recall through their harmonic mean.

Macro-F1 gives each category equal weight. Weighted-F1 weights categories by their number of examples. A confusion matrix places reference labels on rows and predicted labels on columns; off-diagonal counts show mistakes.

Strict entity F1 requires an exact span and type match. Correctly finding “Alex” when the gold span is “Alex Lee” is not a correct full-span prediction.

Uncertainty and calibration

The report includes a 95% accuracy interval from 400 report-level bootstrap resamples. It captures variation when resampling this synthetic test, not uncertainty from new templates, new sources or new cities.

A score of 0.9 is useful only if similarly scored predictions are correct roughly 90% of the time. Temperature scaling adjusts TF-IDF probabilities using validation labels. It preserves the winning category and changes score sharpness.

Expected Calibration Error (ECE) compares average confidence with accuracy in bins. Lower is generally better under the same evaluation. Calibration can become worse when the data distribution changes, so before/after results are reported honestly.

Predictions below the configured 0.60 score are marked for review. The app also flags fewer than three words, low training-vocabulary coverage, raw confidence below 0.45 or a raw top-two margin below 0.15. These are transparent heuristics, not a separate model. Reported calibration coverage measures only the 0.60 calibrated-score threshold, not the combined UI policy. Accuracy among accepted predictions must be read alongside coverage: accepting very few easy cases can inflate that accuracy.

Comparing versions

V1 artifacts are retained in `artifacts/baselines/v1/`. V2 changes the corpus, entities, splits and training protocol. A higher V2 NER score therefore does not isolate the benefit of one algorithm. It is not an apples-to-apples causal improvement claim. The measured appendix gives all current model scores, including disappointing results.

Historical V2 results (archived; not the current benchmark)

Word2Vec document classification reaches 95.17% accuracy on the fixed final sample, compared with 83.67% for TF-IDF and 83.25% for compact BERT. This is evidence about this particular experiment, not a universal model ranking.

TF-IDF's training-only grouped cross-validation macro-F1 is only 0.2502, much lower than its 0.7934 final-test macro-F1. These different held-out variant sets have very different difficulty. The disparity warns that the final split alone gives an incomplete picture of robustness.

CRF's overall strict entity F1 is 0.9846, but PROPERTY recall is only 0.44 and its F1 is 0.6111. More frequent, easier entity types dominate the overall score. This is why a useful evaluation shows per-type results, not just one impressive headline number. BERT entity F1 is 0.8741 on the final sample.

TF-IDF calibration lowers final-test ECE from 0.3830 to 0.0757 in this run. At the 0.60 threshold, calibrated coverage is 92.17% and accepted accuracy is 85.80%. These values describe confidence behavior on the synthetic sample; they do not certify real-world correctness.

Syntax, polarity, threat cues, summaries and retrieval do not yet have independent human-labeled evaluation sets. Do not invent accuracy numbers for them.

11. Dashboard: a guided tour

1. **Overview:** filter synthetic counts by category, district and reporting period. Trends describe the generator, not a real city's crime rate.
2. **Case explorer:** choose Keyword, Semantic or Hybrid search; filter predicted entity types or values; move between result pages; open a case.
3. **Narrative lab:** paste English text or upload UTF-8 .txt, choose Automatic/CRF/BERT entity extraction, and analyze. The limit is 30,000 characters.
4. **Model evaluation:** compare classifiers, inspect confusion matrices, entity scores, calibration and training history.
5. **Topics:** inspect top words, temporal counts, example reports and the NMF comparison.
6. **Pipeline & syllabus:** inspect cleaning counts, Spark execution evidence and syllabus mapping.
7. **Learning guide:** read the short explanation directly inside the app.

Within an analysis, the tabs show entity highlights and summaries, grammatical dependencies, source-sentence evidence search, and prediction details. JSON exports contain the full analysis. The separate redacted-text download masks detected people and contact details but can miss information.

The overview loads metadata columns. Full narrative text is fetched from SQLite for selected records. CSV export applies to the visible result page, avoiding an unexpectedly huge download.

The optional shared password is set through `CRIME_DASHBOARD_PASSWORD`. It is a basic local access gate, not SSO, per-user roles or a production authentication system. Keep the demonstration bound to localhost unless deploying it with appropriate hosting controls.

User-submitted text is analyzed in memory. Saving a correction is an explicit action; the local feedback database stores its text hash, predicted class and corrected class. It does not automatically retrain a model. Clear the session to discard its in-memory analysis.

12. Run the project step by step

On the completed development computer

Open the project folder, run `START_DASHBOARD.cmd`, and visit http://localhost:8502. Read the root README, not `.pytest_cache/README.md`: that cache file belongs to the testing tool.

A fresh Windows installation

Install Python 3.12 and Java 17. Use a trusted compatible Hadoop native distribution on Windows and set HADOOP_HOME if needed. The supplied setup script can prepare the local Java and Python resources; Linux avoids Windows Hadoop native-library issues. Allow several GB for packages, data and downloaded models.

Open PowerShell in the repository folder:

```
py -3.12 -m venv .venv312
.\.venv312\Scripts\python.exe -m pip install -r requirements.txt
.\.venv312\Scripts\python.exe scripts\download_resources.py
.\.venv312\Scripts\python.exe run_pipeline.py
.\.venv312\Scripts\python.exe -m pytest -q
.\.venv312\Scripts\python.exe -m streamlit run app.py
```

A virtual environment is a project-specific collection of Python packages. Using its full Python path ensures that installation and execution use the same interpreter.

The first resource download needs internet. Ordinary inference uses local models afterwards. The GitHub repository excludes large generated data, weights and runtimes; a fresh clone must rebuild them.

On Linux, create a Python 3.12 virtual environment, activate it, install Java 17 and the requirements, then run the same scripts using `python`. The separate native standalone launcher is intended for the tested Windows host; the provided container configuration is an unverified deployment option.

Run separate stages

```
.\.venv312\Scripts\python.exe run_pipeline.py --stage generate
.\.venv312\Scripts\python.exe scripts\run_cluster.py
.\.venv312\Scripts\python.exe run_pipeline.py --stage train
.\.venv312\Scripts\python.exe run_pipeline.py --stage enrich
```

The cluster launcher runs the Spark stage against its master and two workers, then stops only processes it started. Do not run another Spark rebuild concurrently.

`--resume` reuses a stage only when recorded source, settings, inputs and output hashes match. Hashing is conservative and can take time. Changes to source can invalidate earlier stages even when a narrower dependency analysis would permit reuse.

Stop the dashboard before rebuilding. Upstream changes require downstream rebuilding. Do not retrain merely to obtain a more flattering final-test score.

Troubleshooting

Symptom	What to check
Module not found	Use .venv312 Python and install requirements there
Java gateway fails	Java 17, JAVA_HOME, available ports and Windows Hadoop libraries
Dashboard shows setup	Finish enrichment and check its log; all V2 indexes must exist
Address already in use	An existing dashboard may already be running on port 8501
Missing pretrained files	Run download_resources.py with network access
No search matches	Broaden filters; Keyword search requires vocabulary overlap
Model score seems wrong	Inspect the narrative and model limitations; never silently relabel gold data
PDF still looks old in the IDE	Close and reopen the regenerated PROJECT_REPORT.pdf

13. Code and artifact tour

File or folder	Responsibility
settings.toml	Seeds, training sizes, thresholds and Spark defaults
run_pipeline.py	Stage ordering, arguments, hashes and run receipts
crime_nlp/generate.py, corpus_patterns.py	Fictional corpus and reference spans
scala/CrimePipeline.scala	Main Scala Spark cleaning, SQL, quality reports and MLlib
crime_nlp/spark_pipeline.py / scala_runner.py	Engine selection, original PySpark implementation and Scala compilation/submission
crime_nlp/train.py	Shared splits, classical models, topics and final scoring
crime_nlp/bert_model.py, ner_model.py	BERT document and token classification
crime_nlp/evaluation.py	Metrics, calibration and BIO repair
crime_nlp/inference.py	Assemble one narrative's complete analysis
crime_nlp/text.py, topics.py	Text utilities, summary, cue heuristic and topic preprocessing
crime_nlp/semantic.py	Sentence embeddings, fusion and evidence retrieval
crime_nlp/store.py, privacy.py	SQLite lookup, feedback, mention assertions and redaction
crime_nlp/enrich.py	Batch predictions and search indexes
crime_nlp/audit.py, import_data.py	Hash receipts and external staging validation
app.py	Streamlit pages and interactions
scala/CrimeAnalytics.scala	Scala and Spark Dataset exercise
tests/	Unit, artifact and dashboard checks
artifacts/	Measured metrics, split IDs and reproducibility evidence
docs/	Beginner guide, report, measured results, cards and demonstration

To trace a feature, start with its dashboard control, find the called inference function, then find the saved model it loads and the training function that created it. This is more manageable than reading every file from top to bottom.

Tests check behavior rather than proving real-world model reliability. Unit tests cover offsets, negation, calibration, imports and privacy. Integration tests check 60,000-row output alignment, split separation, model inference and dashboard pages. CI runs lightweight tests without rebuilding the full dataset.

14. Syllabus coverage and the Scala example

The detailed table is in SYLLABUS_MAPPING.md. NLP foundations appear in tokenization, morphology and parsing; representations in BoW, TF-IDF and neural embeddings; sequence models in BERT and CRF; applications in NER, sentiment and summarization.

Big Data topics appear in Spark partitions and execution, relational DataFrames and SQL, transformations/actions, Parquet and MLlib pipelines. The Scala file demonstrates immutable values, collections, case classes, traits, pattern matching, closures and map/filter-style operations.

BERT and CRF are chosen from the requested alternatives. GPT, HMM, machine translation, GloVe, streaming ingestion and a generative n-gram language model are not additional implemented systems. Bigrams in a vectorizer are features, not a probabilistic text generator.

Run `python scripts/run_scala.py` inside the environment to execute the Scala example against the processed data. The report does not imply that the main Python dashboard was rewritten in Scala.

15. What improved across versions?

Version 3 broadens the data to informal English and international names, preserves spans through real noise edits, uses word and character TF-IDF in the existing classifier, expands the CRF context features and retrains the original model families. It adds direct name/role checks, explicit input-quality review reasons and a hand-written development comparison against retained V2 weights. ROBUSTNESS_V3.md documents the constraints and remaining limits. No external spell checker, translation service or new model family was added.

The earlier Version 2 changes are retained:

The upgrade adds varied composite scenarios, eight entity types, a separately fine-tuned BERT entity tagger, validation-based tagger selection, a final-test freeze receipt, grouped cross-validation, bootstrap intervals, calibration and review flags.

It also adds cleaner topic inputs and NMF comparison; real two-worker single-host Spark execution and quality reports; semantic/hybrid retrieval and cited evidence sentences; a disk-backed case store and pagination; prediction explanations, redacted downloads and opt-in corrections; a validated external import staging path; and this beginner documentation.

These are implemented features, not guarantees that all scores increased. Refer to IMPROVEMENTS_V2.md for the distinction between delivered work and future extensions.

16. Limits and useful future work

The highest-value next experiment is an independently annotated, permissioned real narrative dataset with double annotation and disagreement review. It should have a source-separated test set, not a random split of near-duplicate stories.

Other future work includes human judgments for retrieval and summaries, explicit false-negative analysis for redaction, multilingual evaluation, temporal drift monitoring, and calibrated entity scores. Scalability work should measure throughput and memory across physical machines, then choose a distributed store or approximate nearest-neighbor index based on observed need.

Streaming, HDFS, cloud deployment, OAuth/RBAC and centralized audit retention require additional infrastructure and are not claimed complete. A synthetic demonstration should not be used to rank real individuals, predict criminality or automate consequential decisions.

17. Glossary and viva questions

Term	Plain-language meaning
Corpus	A collection of texts
Feature	A numerical signal used by a model
Label	The answer a supervised model learns to predict
Embedding	A dense learned vector representing text
Epoch	One pass through training examples
Hyperparameter	A setting chosen outside the fitted model weights
Overfitting	Learning training-specific patterns that fail on new data
Leakage	Evaluation information influencing training or selection
Inference	Running a trained model on input
Calibration	Aligning confidence scores with observed correctness
NER	Finding typed mentions inside text
Partition	One portion of a distributed dataset
Checkpoint	Saved model parameters
Provenance	Where data came from and how it was processed
Hash	A fingerprint used to detect content changes

Why compare BoW with BERT? To test whether contextual neural complexity helps under the same data protocol, instead of assuming that it must.

Why preserve the raw narrative? So highlights and source citations remain anchored to the exact input.

Why use CRF alongside BERT? They provide different sequence-modeling approaches; validation determines which works better here.

Does high synthetic accuracy prove deployment readiness? No. Shared generation patterns and missing real-source evaluation remain major limitations.

Why not call sentiment a danger score? Polarity measures emotional language, while danger needs a different labeled task and validation.

Why have both Parquet and SQLite? Parquet serves batch analytics and selected-column reads; SQLite serves small indexed case lookups and local feedback.

What proves Spark ran? Saved execution metrics, the physical plan, standalone worker registration and successful output invariants—not just importing PySpark.

References and further reading

- Apache Spark 3.5.5 documentation — architecture, SQL and MLlib.
- BERT paper — contextual transformer pretraining.
- Hugging Face token classification — subword/label alignment.
- MiniLM sentence encoder model card — local semantic representation.
- Word2Vec paper — neural word representations.
- scikit-learn calibration documentation — interpreting classifier probabilities.
- spaCy linguistic features — syntax and morphology.
- TextRank paper — graph-based ranking for text.
- NLTK VADER documentation — lexicon-based polarity analysis.

Executed project results

Generated from saved artifacts; all scores below are measured, not illustrative.

Data and execution

- Raw reports: **62,700**
- Clean unique reports: **60,000**
- Invalid rows removed: **1,200**
- Duplicates removed: **1,500**
- Spark: **3.5.5**, master **spark://127.0.0.1:17077**, **8** input partitions
- Enriched narratives: **60,000**

Same-split model comparison

Train / validation / development / final test: 12,000 / 800 / 800 / 1,200.

Model	Accuracy	Macro F1	Weighted F1
Bag of Words	0.8350	0.8361	0.8361
TF-IDF	0.9242	0.9225	0.9225
Word2Vec	0.8217	0.8188	0.8188
BERT	0.8167	0.8139	0.8139

Repeat evaluation on previously viewed synthetic cases. BoW, TF-IDF, BERT and CRF candidates were selected on validation. Word2Vec deployment retained its baseline after observing a diagnostic test regression. This release choice used test results; it is not an untouched final benchmark. No further training or tuning was performed.

Entity extraction

Validation-selected CRF strict entity F1: **0.9223** on 600 evaluation narratives.

Previously viewed synthetic test subset plus generated role/context cases sharing grammar; diagnostic evaluation, not untouched real-source validation.

Entity	Precision	Recall	F1
SUSPECT	0.9241	0.8134	0.8652
VICTIM	0.8891	0.8191	0.8527
LOCATION	0.9597	0.8389	0.8953
WEAPON	1.0000	0.9861	0.9930
PROPERTY	0.9778	0.9075	0.9413
DATE	1.0000	1.0000	1.0000
TIME	1.0000	1.0000	1.0000
EVIDENCE	1.0000	1.0000	1.0000

Distributed baseline and topics

MLlib accuracy: **0.7970**; weighted F1: **0.7895**. It uses 39,528 training and 6,824 test rows, so sample sizes differ from the four-model comparison.

LDA held-out perplexity: **476.67**; top-term diversity: **0.900**.

Verification

Pytest: **281 passed, 0 failed, 0 errors, 0 skipped**. Source: artifacts/spark_backend_regressions.json.

Scala report count verified: **60,000**.

The Windows Scala run completed its computations and exited successfully, with a non-fatal Spark temporary-JAR cleanup warning at shutdown.

Current browser feature audit: **passed**, with **12/12** workflows passed. See docs/FEATURE_AUDIT.md for the full feature matrix and remaining limitations.

Classification uncertainty and calibration

Model	95% accuracy interval
Bag of Words	0.8142 to 0.8558
TF-IDF	0.9083 to 0.9392
Word2Vec	0.8000 to 0.8425
BERT	0.7958 to 0.8392

Intervals use 400 report-level bootstrap resamples and do not cover new-source uncertainty.

Training-only grouped TF-IDF CV mean macro-F1: **0.6831**.

TF-IDF final-test calibration	ECE	Log loss	Coverage at 0.60	Accepted accuracy
before	0.2826	0.5516	0.5617	0.9970
after	0.0332	0.2195	0.9683	0.9363

Temperature is fitted on validation only. Calibration is not guaranteed to improve under a shifted test distribution.

Both entity models

Model	Validation strict F1	Diagnostic evaluation strict F1
CRF	0.9439	0.9223
BERT	0.8537	0.7912

Informal English development challenges

Fixed hand-written development/regression examples; not an independent real-world benchmark.

Release	Category cases correct / 32	Exact people + location cases / 8
v2	23	0
v3	31	8
v3.1	32	8

These hand-written examples informed development. They are not an untouched final benchmark or evidence of flawless real-world performance. Every case is retained in artifacts/robustness_development.json.

Independent score recomputation

Verification: **passed**. Recomputed classification metrics on 1200 saved predictions and entity span F1 on 600 narratives. Frozen model/prediction hashes, reference spans and test IDs were checked. No models were fitted during verification.

Case summaries

All 60,000 enriched cases include a one-line TextRank summary in Parquet, SQLite and case-search exports. One-sentence extractive TextRank, whitespace normalized to one line; source text retained. Summary quality has not been evaluated against human reference summaries.

Initial media evidence correction (historical)

This receipt covers the earlier narrow media-only rule. The current broader context layer is measured separately below.

Narrow disclosed context-rule correction, not CRF retraining. Existing viewed synthetic cases are diagnostic; reported-example tests are development regressions. Raw model weights and benchmark predictions are retained.

The reported Camera footage false-person label is corrected to EVIDENCE by a disclosed context check. Rule corrections have no model probability.

CRF validation Span-F1: 0.9439 before and 0.9439 after. CRF diagnostic Span-F1: 0.9223 before and 0.9223 after. Original model weights and reference labels are unchanged.

Legacy eight-label context diagnostics

Diagnostic on previously viewed synthetic evaluation; WITNESS is separate from the eight trained labels. Raw models and predictions unchanged.

These core/context measurements precede the separately scored ten-label role upgrade below. Witness clauses are separated from locations; context repairs have no model probability. Legacy learned weights remain unchanged.

Diagnostic dataset	Raw Span-F1	Context pipeline Span-F1
CRF validation	0.9439	0.9393
CRF	0.9223	0.9221
BERT	0.7912	0.8063

These are previously viewed synthetic cases. Validation and diagnostic-test changes can differ; the context layer is not an across-the-board benchmark improvement. Some added mentions and article boundaries differ from generator annotations.

Hand-annotated development audit: 38/38 cases meet their specified requirements. Separate variations: 17/17. These examples informed repairs and are not a real-world accuracy estimate. See docs/NARRATIVE_VALIDATION.md.

Stored entity structural validation: **passed** across 60,000 reports; 0 source-offset/overlap failures. This checks data integrity, not semantic correctness.

Petrol and material roles

Automatic entity pipeline: 37/37 material-role development checks passed. Petrol used to set a shop on fire is highlighted as WEAPON (accelerant); theft and forensic collection contexts are distinguished. Corrections have no model probability. These are development checks, not an independent accuracy estimate. See docs/MATERIAL_HIGHLIGHTING.md for scope and diagnostic tradeoffs.

BERT with context layer: 37/37 material-role checks passed.

Validation-selected accuracy revision

Repeat evaluation on previously viewed synthetic cases. BoW, TF-IDF, BERT and CRF candidates were selected on validation. Word2Vec deployment retained its baseline after observing a diagnostic test regression. This release choice used test results; it is not an untouched final benchmark. No further training or tuning was performed.

Model	Previous accuracy	Current accuracy	Change (percentage points)
Bag of Words	0.8067	0.8350	+2.83
TF-IDF	0.8825	0.9242	+4.17
Word2Vec	0.8217	0.8217	+0.00
BERT	0.7958	0.8167	+2.08

Entity model	Previous strict Span-F1	Current strict Span-F1	Change (percentage points)
CRF	0.9178	0.9223	+0.45
BERT	0.7912	0.7912	+0.00

Rejected Word2Vec experiment: validation-selected candidate accuracy was 75.75%, versus 82.17% for the baseline. The baseline is retained for deployment. This release rollback used the viewed test results; no new fitting or tuning followed. Candidate evidence is preserved under artifacts/experiments/accuracy_20261107.

Training sizes and evaluation cases are unchanged. Model tuning used validation; any test-informed release rollback is disclosed above. The viewed synthetic evaluation does not establish real-world accuracy. See docs/ACCURACY_IMPROVEMENTS.md for recipes and reproduction.

Schema-4 generalization upgrade

Diagnostic comparison on previously viewed synthetic ten-label annotations; same current core/context rules in both arms

Same ten-label diagnostic	Strict Span-F1
without trained roles	0.5921
with trained roles	0.6230

The selected CRF role specialist adds ORGANIZATION/WITNESS while preserving the legacy core. These task-specific results are not comparable with the legacy eight-label table.

Role-specialist validation F1: 0.7729; diagnostic F1: 0.5410. Its high precision and limited recall are documented in UPGRADE_STATUS.md.

Windowed BERT candidate validation macro-F1: 0.9246, versus retained 0.9270. The candidate was rejected. Full-document inference with the retained weights scores 0.8167 on the viewed diagnostic test; no classification accuracy gain is claimed.

Source-window retrieval: 61,300 windows cover 60,000 reports. Source text, offsets and ordering were verified. The dashboard includes entity correction, opt-in annotation drafts, qualified action/reference notes and per-entity review.

The annotation contract supports ten labels. Agreement, adjudication and source-separated preparation tools are provided. Thirty-six human quality-review forms remain unreviewed; real-source accuracy and human summary/search/topic quality are not measured. See UPGRADE_STATUS.md, ANNOTATION_GUIDELINES_V4.md and REAL_DATA_WORKFLOW.md.

Interpretation

The corpus is synthetic. Scenario groups are separated, but vocabulary and generator conventions are shared. Model and prediction hashes record the release state; repeat evaluation, prediction reuse and any test-informed release rollback are disclosed in the evaluation receipts and above. No human-reference accuracy is claimed for syntax, sentiment, threat heuristics, search or summarization. Standalone Spark master and worker processes; see cluster_execution.json for verified workers and host scope. Docker and multi-host deployment were not exercised. Version benchmarks use different corpora and protocols.

Resources: `artifacts/evaluation.json`, `ner_evaluation.json`, `spark_metrics.json`, `topics.json`, split ID CSVs, and `logs/`.

Scala Spark implementation verification

The main Spark stage is Scala 2.12.18 / Spark 3.5.5. Comparison status: **passed**, with 3 fresh-process full-corpus runs per backend.

All cleaned fields, SQL trends, quality reports, held-out MLlib labels and probability vectors matched. Fresh NLP inference reproduced the saved classification and entity scores; retained model hashes were unchanged.

Local median end-to-end runtime: PySpark **25.722 seconds**, Scala **29.549 seconds** (+14.88% for Scala). Scala compilation on every invocation is included; no accuracy or end-to-end speed improvement is claimed.

See docs/SPARK_BACKEND_COMPARISON.md and artifacts/spark_backend_comparison.json for complete results. Historical dataset and cluster receipts retain their original run provenance.